Homework 7

Propositional Logic Calculator Front-end

CS3490: Programming Languages

1. Reading

- Read the first half of Chapter 7 of the Haskell tutorial, at
  `https://learnyouahaskell.github.io/modules.html`
  This covers the first three sections: loading modules, Data.List, and Data.Char.
  Make a list of 10 functions from the List and Char library that you think will be most useful for writing programs that work with structured data.
- Read the first half of Chapter 9 of the Haskell tutorial, at
  `https://learnyouahaskell.github.io/input-and-output.html`
  This covers the first two sections: "Hello world", Files and Streams.
  Make sure you pay particular attention to the behavior of the `IO` type wrapper and the `do` notation.
  For the terminal interfaces we will be doing in this class, the following I/O functions will generally suffice:
  `putStr, putStrLn, readFile, writeFile`

2. Extending the syntax of boolean formulas

In this assignment, you will work with an extension of the datatype of Propositions that now includes new operations of logical implication, equivalence (if-and-only-if), and exclusive or.

The set of these formulas is defined by the following grammar:

$$\mathbb{P} \quad ::= \quad \mathbb{V} \mid \mathbb{B} \mid \mathbb{P} \wedge \mathbb{P} \mid \mathbb{P} \vee \mathbb{P} \mid \neg \mathbb{P} \mid \mathbb{P} \rightarrow \mathbb{P} \mid \mathbb{P} \leftrightarrow \mathbb{P} \mid \mathbb{P} \oplus \mathbb{P}$$

This grammar is represented in Haskell using the datatype

```
type Vars = String
data Prop = Var Vars | Const Bool | And Prop Prop | Or Prop Prop | Not Prop
                  | Imp Prop Prop | Iff Prop Prop | Xor Prop Prop
  deriving (Show,Eq)

prop1 = Var "X" `And` Var "Y"                    --    X /\ Y
prop2 = Var "X" `Imp` Var "Y"                    --    X -> Y
prop3 = Not (Var "X") `Or` (Var "Y")             --   !X \/ Y
prop4 = Not (Var "X") `Iff` Not (Var "Y")        --  !X <-> !Y
```

You are allowed to use solutions to the previous homework on propositional logic in solving the following problems.

2.1. Variables

Extend the *free variables* function to account for the new operators.

```
fv :: Prop -> [Vars]
fv prop2 = ["X","Y"]
fv prop4 = ["X","Y"]
```

2.2. Evaluator

Extend the evaluation function to account for the new operators.

```
type Value = Bool
type Env = [(Vars,Value)]
eval :: Env -> Prop -> Value
```

For negation, conjunction, and disjunction, you should use the standard truth tables to define the meaning of these operations. (As you did in the last assignment.) The truth tables for the new operations of implication, equivalence, and eXclusive OR are given below:

X	Y	$X \to Y$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\bot$
$\bot$	$\top$	$\top$
$\bot$	$\bot$	$\top$

X	Y	$X \leftrightarrow Y$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\bot$
$\bot$	$\top$	$\bot$
$\bot$	$\bot$	$\top$

X	Y	$X \oplus Y$
$\top$	$\top$	$\bot$
$\top$	$\bot$	$\top$
$\bot$	$\top$	$\top$
$\bot$	$\bot$	$\bot$

```
eval [("X",False),("Y",True)]  prop2 = True
eval [("X",True) ,("Y",False)] prop2 = False
eval [("X",False),("Y",True)]  prop4 = False
eval [("X",False),("Y",False)] prop4 = True
```

3. Classification of formulas

Recall the following classification of formulas in propositional logic.

- A formula is *satisfiable* if there *exists* a variable environment making the formula true.
- A formula is a *contradiction* if *no* variable environment makes the formula true.
- A formula is a *tautology* if *every* variable environment makes the formula true.
- A formula is a *contingency* if some environments make it true, and some make it false: it is *neither* a contradiction *nor* a tautology.

In the last homework, you implemented the function `sat :: Prop -> Bool` that determines whether a formula is satisfiable.

In the next few problems, you are asked to similarly implement functions that check whether a formula is a tautology or a contradiction. You will also be asked to find a satisfying/contradicting environment, if it exists.

You can use the functions from the previous homework: `evalList`, `genEnvs`, `sat`, etc.

Hint. There are several approaches to the problems that follow. The direct route is to construct a truth table and evaluate the formula in each possible environment, similarly to how you implemented `sat` in the previous assignment.

But you can also use boolean algebra/logic to reduce the checking of tautology/contradiction to the previously defined function `sat`.

3.1. Checking contradictions and tautologies

Implement the following functions

1. `contra :: Prop -> Bool` determines whether the formula is a contradiction.

   ```
   contra (Iff (Var "X") (Var "Y"))       = False
   contra (Iff (Var "X") (Not (Var "X"))) = True
   ```

2. `tauto :: Prop -> Bool` determines whether the formula is a tautology

   ```
   tauto (And (Var "X") (Var "X")) = False
   tauto (Imp (Var "X") (Var "X")) = True
   ```

3.2. Finding satisfying/refuting assignments

Implement the following functions

1. `findSat :: Prop -> Maybe Env` which, given a formula, outputs a variable assignment that makes the formula true. If no such assignment exists, it should output `Nothing`.

   ```
   findSat (Iff (Var "X") (Var "Y"))       = Just [("X",True),("Y",True)]
   findSat (Iff (Var "X") (Not (Var "X"))) = Nothing
   ```

2. `findRefute :: Prop -> Maybe Env` which, given a formula, outputs a variable assignment that makes the formula false. If no such assignment exists, it should output `Nothing`.

   ```
   findRefute (And (Var "X") (Var "X")) = Just [("X",False)]
   findRefute (Imp (Var "X") (Var "X")) = Nothing
   ```

3.3. Classifying formulas

Write a function `classify :: Prop -> String` which acts as follows.

- If the formula is a tautology, output `"tautology"`
- If the formula is a contradiction, output `"contradiction"`
- If the formula is a contingency, output `"contingency"`

Hint. Use a stack of guards akin to a `switch` statement in C/Java.

4. Checking logical equivalence

Recall that two boolean formulas `p1` and `p2` are *logically equivalent* if they evaluate to the same truth value in every possible environment.

In terms of truth tables, this means that when you build the truth table for both formulas, they yield the same output for every row of the table.

Implement the following functions.

Hint. You can either attack these problems directly by generating the "truth table", or you can realize that two formulas are equivalent if a certain formula built from them is a tautology.

1. `checkEq :: Prop -> Prop -> Bool` outputs `True` if the two given formulas are logically equivalent.

   ```
   checkEq prop1 prop2 = False
   checkEq prop2 prop3 = True
   checkEq (Imp (Var "X") (Var "X")) (Const True) = True
   checkEq (Imp (Var "X") (Var "Y")) (Const True) = False
   ```

2. `refuteEq :: Prop -> Prop -> Maybe Env` outputs `Just e` where `e` is an environment under which the two given formulas evaluate to *different* values, if it exists. Otherwise, it should output `Nothing`.

   ```
   refuteEq prop1 prop2 = Just [("X",False),("Y",False)]
   refuteEq prop2 prop3 = Nothing
   refuteEq (Imp (Var "X") (Var "X")) (Const True) = Nothing
   refuteEq (Imp (Var "X") (Var "Y")) (Const True) = Just [("X",True),("Y",False)]
   ```

5. Lexical Analysis

Define the following datatypes for operators and tokens.

```
-- binary operators
data BOps = AndOp | OrOp | ImpOp | IffOp | XorOp
  deriving (Show,Eq)

-- the type of tokens
data Token = VSym Vars | CSym Bool | BOp BOps | NotOp | LPar | RPar
           | Err String -- auxiliary token to store unrecognized symbols
           | PB Prop    -- auxiliary token to store parsed boolean expressions
  deriving (Show,Eq)
```

Write a function `lexer :: String -> [Token]` which converts a string to a list of tokens.

The lexical rules are as follows.

Variables are defined by any string starting with a capital letter and followed by zero or more alphanumeric characters. For example, the following are variables:

```
X  Y  Z  X1  Y12  ZZZ  A  B  P  Qo01j5
```

Operators and Constants are defined by the concrete syntax given in the following table:

`Prop` Constructor	Token	Concrete Syntax
Not	`NotOp`	`!`
And	`BOp AndOp`	`/\`
Or	`BOp OrOp`	`\/`
Imp	`BOp ImpOp`	`->`
Iff	`BOp IffOp`	`<->`
Xor	`BOp XorOp`	`<+>`
Const True	`CSym True`	`tt`
Const False	`CSym False`	`ff`

Caution!! In Haskell, the backspace character `\` must be escaped at the level of strings. For example, to match a string beginning with `/\`, you should write `lexer ('/':'\\':s) = ...`

```
GHCI> putStrLn "X \\/ Y"
X \/ Y
```

Internally, the string `"/\\"` is still a list containg two elements: the character `/`, represented `'/'`, and the character `\`, represented `'\\'`. (The backslash is *not* escaped when the string is entered by the user via `getLine` function or read from a file via `readFile`.)

Lexer Example

```
lexer "! (X \\/ Y)" = [NotOp, LPar, VSym "X", BOp OrOp, VSym "Y", RPar]
```

6. Parser

Modify the parser for arithmetic expressions implemented during the last lecture so that it parses logical expressions instead.

6.1. Shift-reduce helper function

Implement the function `sr :: [Token] -> [Token] -> [Token]` that applies every matching grammar rule in reverse, shifting the next token from the input to the parse stack if no rules are applicable.

The function should return the stack if no rules are applicable and the input is empty.

6.2. Parsing with error handling

Write the function `parseProp :: String -> Either Prop String` that takes as input a list of tokens and attempts to produce an element of the `Prop` datatype.

If the parse is successful, the result should be placed inside the `Left` branch.

If the parse is not successful, the function should return `Right e`, where `e` is a string identifying the type of error that occurred.

You can use a variation of the shift–reduce parser `sr` (to be implemented in class on Monday).

```
parseProp "!(X\\/Y)"
  = Left (Not (Or (Var "X") (Var "Y")))
parseProp "!(X$Y)"
  = Right "Lexical error: $Y)"
parseProp "!X!Y"
  = Right "Parse error: [PB (Not (Var "Y")),PB (Not (Var "X"))]"
```

7. An front-end with terminal I/O

Write a function `main :: IO ()` which repeatedly queries a user to input a string. It should then attempt to parse the string as a boolean formula, returning an error message *without crashing the program* whenever the string is invalid. If the string is valid, it should classify the string according to whether it's a a tautology, a contradiction, or a contingency. In the latter case, it should also print out a satisfying and a refuting variable assignment.

When the user enters the string `quit`, the program should stop.

A sample interaction could proceed as follows.

```
GHCI> main
X \/ Y
Contingency
[("X",False),("Y",True)]
[("X",False),("Y",False)]
X <+> X
Contradiction
X \/ !X
Tautology
quit
```